Context Compression Techniques for Legal Document Analysis

Sigvard Bratlie

March 27, 2026

Abstract

Legal application are bound to the handling of large scale docuemnt analysis when starting a case. The frontier models that powers the majority of LLM applications are reporting on increasing maximum sequnece length / context windows. But the

Chapter 1

Introduction

The legal industry is increasingly adopting large language models (LLMs) to assist lawyers in navigating the complexity of modern litigation. Legal cases are inherently dynamic: they evolve over time as new documents, correspondence, evidence, and procedural developments continuously emerge. What was once a central claim may become peripheral; a newly submitted document may reframe the entire dispute. This progression means that not only does the volume of material grow, but the *relevance and weight* of existing information shifts as the case develops.

A fundamental limitation of current LLM-based systems is that they are stateless. Without explicit mechanisms for maintaining case state, models must re-read the full document history for every query. More critically, even models that handle large contexts tend to perform well in early stages but *drift* over extended, multi-turn interactions — gradually losing track of what is current, what has been superseded, and what actually matters. In legal case management, this drift is not merely an inconvenience; it can lead to factually incorrect summaries, missed deadlines, or misrepresentation of parties and claims.

This thesis addresses this problem through two complementary mechanisms: *compression* of case-relevant information into a structured representation, and *state maintenance* — the continuous, structured tracking of case essentials as the project evolves. The vehicle for this is a structured *FactSheet*: a living, machine-readable summary of the case that is initialized from initial input and updated incrementally as new documents and information arrive.

The research investigates whether maintaining an explicit, structured state representation improves an LLM agent's ability to preserve contextual accuracy over the full lifecycle of a legal case — particularly as conversation length grows and the document landscape changes. The system is implemented as a LangGraph-based conversational agent, incorporating multi-phase document processing pipelines for both project initialization and incremental updates.

The contributions of this thesis are: (1) the design and implementation of initialization and update pipelines for structured legal case state management, (2) an evaluation framework comparing structured state agents against RAG-only and baseline approaches, and (3) an analysis of accuracy degradation, compression efficiency, and state consistency across extended multi-turn conversations.

The remaining chapters are organized as follows: Chapter 2 presents background and related work on long-context LLMs, context compression, and retrieval-augmented generation. Chapter 3 describes the methodology, system architecture, and evaluation design. Chapter 4 presents experimental results. Chapter 5 discusses findings and implications, and Chapter 6 concludes with a summary and directions for future work.

Chapter 2

Theory

This chapter presents the theoretical foundation for the approaches explored in this thesis. We examine the core challenges of applying large language models to long-context, multi-turn legal document analysis, and survey the relevant literature on retrieval, compression, and agentic context management strategies.

2.1 Large Language Models and Context Windows

Modern large language models (LLMs) are built on the transformer architecture [VSP+23], which processes input as a sequence of tokens bounded by a fixed context window. While recent models advertise longer context windows, empirical benchmarks reveal a significant gap between nominal and effective context utilization. introduce RULER, a benchmark designed to measure the *real* usable context size of long-context LLMs, demonstrating that performance degrades substantially before the advertised limit is reached [HSK+24]. This has direct implications for legal use cases, where case files can span hundreds of pages across dozens of documents.

2.2 The Lost in the Middle Problem

Even within the effective context window, LLMs do not attend uniformly to all input. Liu et al. show that model performance is highest when relevant information appears at the beginning or end of the context, and degrades for information placed in the middle — a phenomenon they term *lost in the middle* [LLH+23]. This positional bias has critical consequences for legal document analysis, where key facts may be embedded deep within lengthy attachments. It motivates the need for strategies that either restructure context placement or reduce reliance on verbatim document injection altogether.

2.3 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) addresses context limitations by retrieving only the most relevant document chunks at query time, rather than loading entire documents into the context window [LPP+21]. A standard RAG pipeline consists of offline chunking and embedding of documents into a vector store, followed by semantic retrieval at inference time. While effective for single-turn question answering, naive RAG struggles in multi-turn conversational settings where relevance depends on accumulated context across turns — a central challenge in this thesis.

RECOMP [XSC23] extends the RAG paradigm with both extractive and abstractive compressors that filter and condense retrieved passages before they are passed to the generator, reducing noise and token consumption while preserving relevant information.

2.4 Prompt Compression

An orthogonal approach to managing long contexts is prompt compression: reducing the token footprint of the input itself rather than selectively retrieving. LLMLingua [JWL+23] proposes a coarse-to-

fine compression method guided by a smaller language model, identifying and removing tokens with low information content. LongLLMLingua [JWL+24] extends this to long-document scenarios, incorporating question-aware compression that prioritizes tokens contextually relevant to the user query. Both methods are relevant as baseline comparisons for the context management strategies implemented in this thesis.

2.5 Graph-Based and Multi-Hop Retrieval

Standard vector similarity retrieval operates on isolated chunks and does not capture relational structure between entities. In legal cases, facts, parties, events, and claims are deeply interconnected, making relational retrieval particularly valuable. GraphRAG [**?**] proposes building a knowledge graph over the document corpus, enabling community-level summarization and structured traversal of entity relationships. HopRAG [LWC+25] further extends this with multi-hop reasoning, constructing logical chains across documents to answer complex queries that require synthesizing information from multiple sources. These approaches are particularly relevant to the factsheet extraction pipeline implemented in this thesis, where structured relationships between parties, events, and claims must be maintained across turns.

2.6 Infinite and Extended Context Architectures

An alternative to prompt-level strategies is modifying the underlying model architecture to support longer effective memory. Munkhdalai et al. propose Infini-Attention, which introduces compressive memory into the transformer attention mechanism to enable theoretically unbounded context without quadratic scaling [MFG24]. While this represents a promising long-term direction, it requires architectural changes to the model and is not applicable in API-based deployment settings, which is the constraint under which this thesis operates.

2.7 Agentic Context Management

Beyond static retrieval and compression, agentic systems can actively manage context across multi-turn conversations. Rolling summarization — periodically condensing conversation history into a compact summary — is a practical strategy for preventing context overflow in long sessions [Mic24]. Stateful agent frameworks such as LangGraph enable persistent state management through checkpointing, allowing the agent to maintain structured representations (e.g., factsheets) that accumulate case knowledge across turns without requiring the full document history to remain in context.

Taken together, the strategies surveyed in this chapter — effective context benchmarking, positional bias awareness, RAG with compression, graph-based retrieval, and agentic summarization — form the theoretical basis for the system design and experimental comparisons presented in the following chapters.

Chapter 3

Method

3.1 System Architecture

The system is designed as a modular, client-server application consisting of four main packages organized as a `uv` workspace: `agent`, `ui`, `evals`, and `data-viewer`. The architecture follows a separation of concerns, where the backend handles all agent logic, data persistence, and document processing, while the frontend is responsible for user interaction and result presentation.

3.1.1 Overview

The backend is implemented as a **FastAPI** application (`agent/main.py`) that exposes a RESTful API with Server-Sent Events (SSE) for real-time response streaming. The frontend is a **Streamlit** application that communicates with the backend over HTTP, consuming streamed responses. Authentication is handled via **Supabase Auth** using JWT tokens, which are validated on each request through a `HTTPBearer` dependency in FastAPI.

3.1.2 Agent Layer

The core agent is implemented as a LangGraph `StateGraph` (see Figure 3.1). The graph has three nodes:

- `init` — Injects the system prompt as the first `SystemMessage` on a new conversation; is a no-op on resumption.
- `call_llm` — Assembles the full input payload (system prompt, factsheet context, conversation history, and any per-request attachment content), sanitizes it for provider compatibility, then streams the LLM response.
- `call_tool` — Executes all tool calls emitted by the LLM in the previous turn, with per-call caching to avoid redundant invocations within the same session.

Control flows as: `START` → `init` → `call_llm`. A conditional edge then routes to `call_tool` if the LLM response contains tool calls (looping back to `call_llm` afterwards), or to `END` if not. To manage long conversations within the model's context window, a rolling summarization strategy is applied every `sum_rate` messages (configurable, default 20) using a dedicated `Summarizer` module.

Conversation state is persisted using LangGraph's `AsyncPostgresSaver` checkpointer, backed by **Supabase PostgreSQL**, enabling session resumption and multi-turn dialogue across requests.

3.1.3 Tool Layer

The agent has access to a set of callable tools defined in `tools.py`. The full tool set (`TOOLS`) used in the primary agent configuration includes:

- `tavily_search` — Web search via the Tavily API.

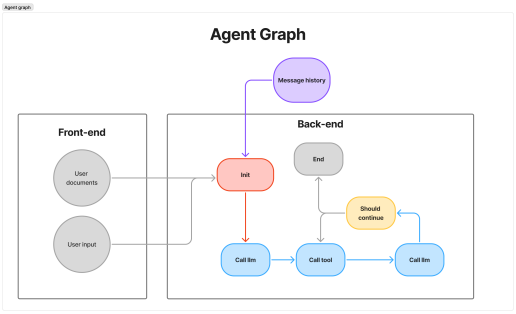

Figure 3.1: Agent graph: `init` → `call_llm` with a conditional loop through `call_tool`.

- `read_attachments` — Fetches and returns the full parsed content of one or more documents by file ID, reading from a database body cache when available and falling back to GCS otherwise.
- `list_attachments` — Lists a project's attachments and emails (with metadata such as title, date, and category), optionally filtered by date range and significance. Returns file IDs that the model can pass to `read_attachments`.
- `show_elements` — Returns structured factsheet elements (events, parties, claims, damages, deadlines) for a project, optionally filtered by date range and significance level.
- `query_project_attachments` — Performs semantic retrieval (RAG) against project-specific document chunks indexed in BigQuery Vector Store.
- `query_laws` — Semantic retrieval against a corpus of Norwegian legislation, with optional filtering by law title.
- `read_specific_law` — Retrieves exact paragraph text from legislation by title and paragraph number via a parameterised BigQuery query.

An ablated tool configuration, `BASELINE_RAG_TOOLS`, is defined for evaluation purposes. It includes web search, law retrieval (`query_laws`, `read_specific_law`), and project document RAG (`query_project_attachments`), but excludes the structured factsheet tools and direct file-reading access.

3.1.4 Data and Storage Layer

Document persistence is handled through two complementary mechanisms. Raw files (PDF, DOCX, PPTX, EML) are stored in **Supabase Storage**, where they can be retrieved on demand by the `read_attachment` tool. Parsed and chunked document content is embedded using **Google's** `gemini-embedding-001` **model** and stored in **BigQuery Vector Store** for semantic retrieval. Structured project data — including the `FactSheet` and session metadata — is stored in Supabase PostgreSQL via the `SupabaseManager` CRUD interface.

3.1.5 Init and Update Pipeline

Project initialization (`POST /init-project`) and updates (`POST /update-project`) are handled by the `ProjectPipeline` class, which compiles a LangGraph `StateGraph` for each operation (see Figure 3.2). The graph exploits LangGraph's fan-out semantics to run independent work in parallel:

- `collapse_emails` and `initialize_input` start in parallel from `START`. `collapse_emails` merges EML thread duplicates; `initialize_input` runs an LLM call to extract an initial structured summary from the user's case description.
- `extract_emails` follows `collapse_emails` and decodes email attachments from the collapsed threads.
- `parsing` and `storage` both fan out from `extract_emails`. `parsing` processes all documents (PDF/OCR, DOCX, PPTX, EML) into plain text; `storage` uploads raw files to GCS in parallel.

- `embedding` follows `parsing` and indexes chunked document content into BigQuery Vector Store.
- `analyze` is a fan-in node that waits for both `parsing` and `initialize_input` to complete. It then dispatches all documents and emails to the LLM in size-bounded batches (see Section 3.2) to extract structured entities: attachments, emails, events, claims, damages, and deadlines.
- `update_metadata` follows `analyze` and runs an LLM call to update the case title and background based on all extracted data.
- `qc_analysis` also fans out from `analyze` and runs a quality-control pass over the extracted entities.
- `save` follows `update_metadata` and persists all extracted entities to Supabase PostgreSQL.

The update pipeline is identical in structure but adds a `load_project_data` node at the start that loads the existing factsheet before analysis, enabling incremental updates. Status events are streamed back to the frontend via SSE at each node transition.

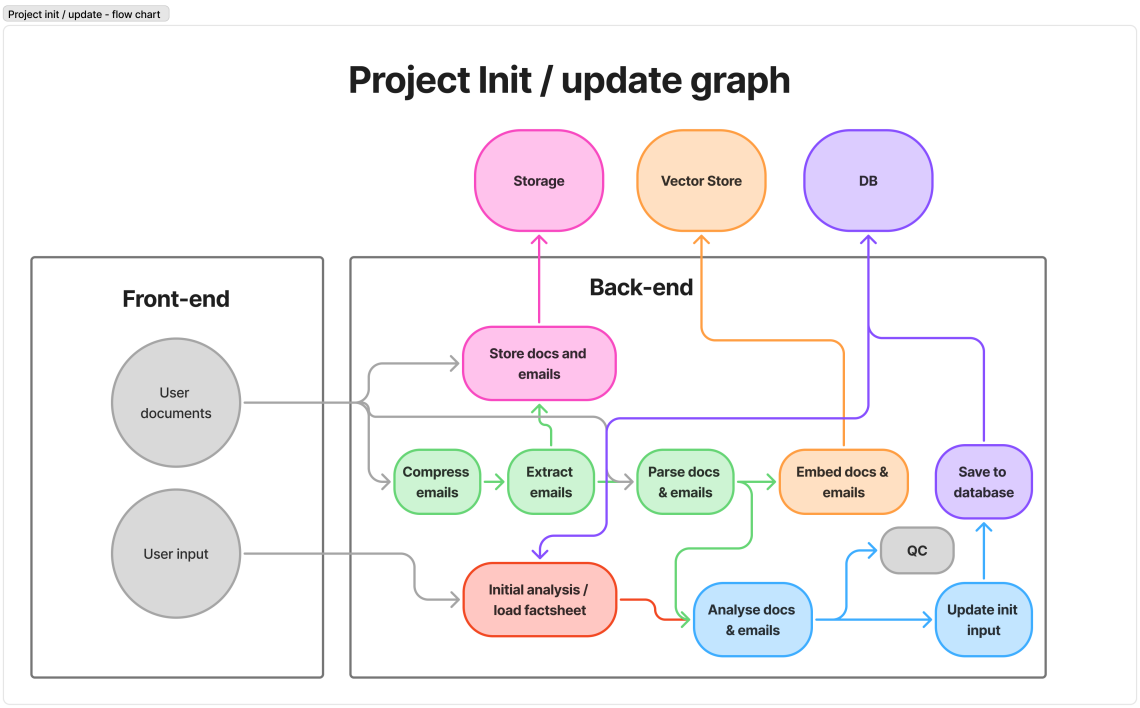

Figure 3.2: Init and update pipeline graph: parallel fan-out for parsing, storage, and initialisation, converging at the `analyze` fan-in node.

3.1.6 Clean Pipeline

The `ProjectClean` class provides two separate cleanup graphs that can be triggered on demand (see Figure 3.3):

- **Clean Elements** (`compile_clean_elements`): `load_project` → `clean` → `save_results`. The `clean` node calls `deduplicate_elements` on the `ContextManager`, which sends all element types (events, parties, claims, damages, deadlines) to the LLM in configurable chunks via `asyncio.gather`. The LLM returns the IDs to keep; all others are discarded. Results are saved back to the database concurrently.
- **Clean Metadata** (`compile_clean_metadata`): `load_project` → `clean` → `save_results`. The `clean` node rewrites the case title and background using the current factsheet content as context. Title and background are saved back concurrently via `asyncio.gather`.

Both graphs share the same linear `StateGraph` structure and reuse the `_load_project_node` and save nodes, differing only in the `clean` step.

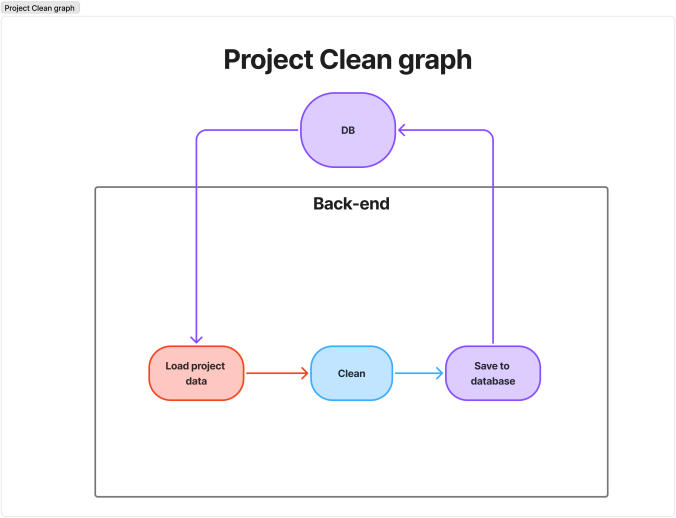

Figure 3.3: Clean pipeline graph: `load_project` → `clean` → `save_results`, used for both element deduplication and metadata rewriting.

3.1.7 LLM Configuration

The system supports multiple LLM providers through LangChain's `init_chat_model` interface. **Google Gemini** serves as the primary provider, with **OpenAI GPT** available as a secondary option. The active model is configurable per session request, allowing for flexible experimentation and evaluation across providers.

3.1.8 Tech Stack Summary

Layer	Technology
Language	Python 3.13
Agent framework	LangGraph (StateGraph)
LLM orchestration	LangChain (`init_chat_model`)
LLM providers	Google Gemini (primary), OpenAI GPT (secondary)
Backend API	FastAPI + Uvicorn
Streaming	Server-Sent Events (SSE)
Frontend	Streamlit
Authentication	Supabase Auth (JWT)
Database	Supabase (PostgreSQL)
Vector store	BigQuery Vector Store (persistent)
File storage	Supabase Storage
Conversation checkpointer	LangGraph `AsyncPostgresSaver`
Document parsing	PyPDF2, ocrmypdf, python-docx, python-pptx
Embeddings	Google `gemini-embedding-001`
Web search	Tavily

Table 3.1: System tech stack.

3.2 Design Choices

The system architecture was guided by several pragmatic design choices, prioritizing development speed, flexibility, and cost-efficiency given the constraints of a master's thesis. By separating the descriptive architecture from the rationale, the core motivations for the selected technologies are outlined below.

Agent Layer and Orchestration

The core agent is implemented using **LangGraph** [Lan24]. This framework was chosen primarily for its fast build time, which is essential given the limited time available for this thesis. It provides a clean `StateGraph` abstraction that is highly expandable for future extensions, such as adding dedicated reasoning nodes or document re-ranking stages. Furthermore, LangGraph is model-agnostic, providing built-in support for multiple LLM providers. This flexibility is crucial, allowing the system to easily switch between models or add new ones as desired for evaluation and experimentation.

Database and Storage Architecture

The system's persistence layer separates structured relational data, unstructured document storage, and semantic vector retrieval to optimize for specific access patterns and constraints.

Relational Storage (PostgreSQL via Supabase): PostgreSQL was selected for its flexibility and highly efficient reads across tables. The structured nature of the factsheet state requires strict enforcement of relationships through primary and foreign keys to maintain data integrity and orderly matching on IDs [Sup20]. An initial attempt was made using a NoSQL database (Firestore); while easy and fast to set up, the lack of an enforced schema led to a loss of overview, with state inconsistencies and errors accumulating over time. Additionally, performing complex reads and combining data across different entities proved expensive and inefficient in NoSQL.

Supabase was chosen as the PostgreSQL provider because it is affordable, open-source, and seamlessly integrates other necessary products like Authentication. Having these features included out-of-the-box makes the development process straightforward and fast.

Database Schema

The database schema is designed to support the agent's state management and document metadata, while being flexible enough to accommodate future extensions. There are three main groupings of tables:

Project related tables
Projects
Parties
Events
Deadlines
Attachments
Emails
Claims (legal specific)
Damages (legal specific)

These tables are combined to form the factsheet state, which is the agent's internal representation of the case. The factsheet is updated and referenced throughout the agent's operation, and it is crucial that it is stored in a structured and consistent manner.

Session related tables
Sessions
Session Events
Attachments

Tables for storing document metadata and relationships.

Agent checkpoints
Checkpoints
Writes
Blobs

Tables for the agent's memory and conversation history.

File Storage (Supabase Storage): Raw documents (PDF, DOCX, EML, etc.) are stored in Supabase Storage. There were no specific architectural requirements for this other than cost-efficiency. Since Supabase was already selected for the PostgreSQL database, using its built-in, affordable object storage was a natural and convenient choice [Sup20].

Vector Store and Embeddings: Semantic retrieval relies on **Google's embedding model** (`gemini-embedding-001`). This model was specifically chosen for its strong multilingual capabilities; smaller and cheaper open-source models often struggle with RAG matching when queries or documents are not strictly in English, which is a critical requirement for handling Norwegian legal texts.

For the vector database, a two-tier approach is used. **ChromaDB** is utilized as an in-memory store for fast, session-scoped retrieval. For persistent, project-scoped retrieval, **BigQuery Vector Search** is employed. BigQuery was chosen because it is affordable and operates on an on-demand pricing model. The primary drawback of BigQuery is its high latency, meaning a significant amount of time is spent indexing and saving to the vector store. However, this trade-off is worthwhile: the alternative would require provisioning an always-running vector database server, which introduces high continuous costs and additional operational setup.

FastAPI and Server-Sent Events

The backend is exposed as a standalone REST API using **FastAPI** [Ram18] to decouple it from the frontend. **Server-Sent Events (SSE)** are used for streaming responses because the agent pipeline is long-running and asynchronous. SSE provides a responsive user experience by pushing incremental status updates and partial results without the complexity of WebSockets, which are unnecessary for this unidirectional communication flow.

Concurrency, Reliability, and Tool Efficiency

Several engineering decisions were made to ensure the system operates efficiently under the latency and throughput constraints imposed by external APIs and large document volumes.

Async-first architecture. The entire backend stack is built on Python's `asyncio` event loop. FastAPI natively supports async request handlers, and all LangGraph pipeline nodes are declared as `async` coroutines. This allows the server to handle multiple concurrent sessions without blocking on I/O-bound operations such as LLM calls, database queries, or storage reads.

Semaphore-based concurrency control. To prevent overloading external services, four independent `asyncio.Semaphore` instances are used — one per resource class: LLM, database, storage, and vector store. Each semaphore has a configurable maximum concurrency value drawn from `config.toml` (e.g. `[async.llm] max_concurrent_requests = 20`). Pipeline nodes acquire the relevant semaphore before dispatching work, ensuring that bursts of parallel tasks are automatically throttled without busy-waiting or sleep loops.

Batch-wise and chunk-wise parallel processing. During project initialization, documents are grouped into batches before being dispatched for LLM analysis. Batching is driven by two configurable thresholds: a maximum byte-size per batch (`project.threshold`) and a maximum file count per batch (`project.max_attachments` / `project.max_emails`). Each batch is wrapped in an async task that acquires the LLM semaphore, and all tasks are dispatched concurrently via `asyncio.gather()`. The same chunk-wise strategy is applied during factsheet cleaning and deduplication: element lists (events, claims, damages, etc.) are split into fixed-size chunks (`project.clean.chunk_size`), each chunk is cleaned by a separate concurrent LLM call, and results are merged after all tasks complete. This approach keeps individual prompts within token limits while maximising throughput.

Rate limiting and 429 handling. To comply with provider-imposed rate limits and avoid HTTP 429 (Too Many Requests) errors, each LLM instance is wrapped with LangChain's `InMemoryRateLimiter`. The limiter implements a token-bucket algorithm: tokens are refilled at a configurable rate (`async.llm.requests_per_seco`

and a burst capacity (`async.llm.max_burst_size`) allows short spikes. A single stateful limiter instance is shared across all coroutines that use the same model, so the rate limit is enforced globally rather than per-request.

Retry mechanism with exponential backoff. Transient API errors (e.g. network timeouts or momentary 5xx responses) are handled by a dedicated `apply_retry()` helper. It wraps any LangChain chain with `with_retry()`, configuring a stop condition (`retry_attempts` from config) and exponential backoff with random jitter (`wait_exponential_jitter=True`). The retry decorator is applied *after* structured-output binding so that validation failures are also covered. If all retry attempts are exhausted, the error is propagated and logged.

TOML-based configuration management. All tunable parameters — concurrency limits, rate limiter settings, retry counts, chunk sizes, model endpoints, and storage bucket names — are centralised in a single `config.toml` file. On startup, the file is parsed into a fully typed `AppConfig` object built with **Pydantic** and `pydantic-settings`. Nested configuration sections map directly to typed dataclasses (e.g. `AsyncConfig`, `ProjectConfig`, `VectorstoreConfig`), providing IDE auto-complete, validation at load time, and a single source of truth for the entire application. A `get_app_config()` singleton cached with `@lru_cache` ensures the config is parsed only once per process.

Tool interface design — string IDs over full objects. Agent tools are designed to accept minimal, flat inputs: all tools receive only string identifiers (e.g. `project_id: str`, `ids: list[str]`) rather than full domain objects. This keeps the JSON schema presented to the LLM small, reduces the token footprint of tool calls, and makes it straightforward for the model to construct valid invocations. Internal data fetching (loading full attachment records, querying the database) is performed inside the tool implementation, hidden from the model entirely.

3.3 Evaluation Design

Describe the evaluation design, including the datasets used, metrics for evaluation, and the experimental setup.

3.3.1 Datasets

The evaluation framework relies on three custom-built datasets developed specifically for this thesis in collaboration with legal experts from the law firm BAHR AS. Each dataset is derived from a unique, real-world legal case processed through the Norwegian district court system. While the final verdicts and formal submissions from both parties were publicly available, the underlying case documents (e.g., correspondence, internal notes, and evidence) remained confidential.

To facilitate a robust evaluation of the RAG (Retrieval-Augmented Generation) system, we have reconstructed these cases by generating synthetic document sets. These sets include simulated emails, technical reports, agreements, and contracts, all meticulously crafted to mirror the factual matrix and legal complexities described in the official court records. This approach ensures that the agent is tested against a realistic representation of a legal case file while maintaining necessary privacy standards.

The datasets are organized to simulate the chronological lifecycle of a legal proceeding. Each case is structured into several *sessions*, each containing a sequence of *queries*. This structure is designed to evaluate two critical capabilities of the agent:

1. **Contextual Awareness**: The ability to maintain a coherent understanding of the case as it evolves through different stages of litigation.

2. **Analytical Accuracy**: Each query is paired with a "gold standard" expected output, defined by legal experts. These references serve as the ground truth for measuring correctness, relevancy, and completeness.

By following a chronological order, the sessions require the agent to handle increasing amounts of information and maintain consistency over time, reflecting the professional requirements of a legal practitioner managing an ongoing case.

3.3.2 Dataset Limitations and Caveats

Several factors limit the generalisability of the evaluation and should be considered when interpreting the results.

Absence of a plain LLM baseline. A natural comparison point would be an ordinary LLM queried without any specialised architecture — no document processing, no vector store, no RAG, simply the raw case files passed as context. This approach was tested and found to be infeasible. The aggregated size of a realistic legal case file — spanning contracts, correspondence, technical reports, and legal submissions — frequently exceeds the usable context window of most commercially available models, and is well beyond the limits of the majority of open-source and cost-efficient alternatives. Feeding entire case files into a plain LLM context is therefore not a viable evaluation strategy, and the minimum meaningful comparison is a RAG-enabled architecture. This is also why the evaluation compares the custom agent against a Baseline + RAG configuration rather than an unaugmented LLM.

Dataset size and the scalability trade-off. The evaluation uses a constrained number of cases and queries, as running a large-scale benchmark across multiple model-agent pairs with repeated trials becomes prohibitively expensive and time-consuming. This is, however, in tension with a fundamental property of the system: the agent is designed to improve with larger and richer document sets, as more material increases the factsheet's coverage and the quality of RAG retrieval. Evaluating on small datasets therefore likely *underestimates* the performance advantage of the custom agent in production-scale deployments.

Heterogeneity between cases. The evaluation datasets differ substantially in complexity. Two of the three cases consist of fully synthetic documents generated to match the factual matrix of published court records, while the third is derived from a real case with a significantly larger document volume. The real case contains no emails (which were excluded to maintain confidentiality), yet it is considerably more complex, covering more parties, a longer timeline, and more contested facts. This asymmetry means that performance differences across cases may reflect document complexity as much as agent capability, and cross-case comparisons should be made with caution.

Limitations of expected-answer evaluation. Model responses are graded against expected answers produced in collaboration with legal experts at BAHR AS. This approach carries two inherent risks: the expected answers themselves may contain inaccuracies, and — more fundamentally — many legal queries are open-ended, such that a correct and complete model response may differ substantially from the reference without being wrong. Three mitigations were applied to reduce this risk:

1. **Fact-based scope**: The evaluation is restricted to factual, document-grounded questions (e.g. "Who are the parties?", "What damages are claimed?"), deliberately excluding normative or predictive legal questions where correct answers are inherently ambiguous.
2. **Validatable content**: Questions are formulated so that their answers can be directly verified against the case documents, reducing the space for legitimate variation.
3. **Minimal-requirement framing**: Expected answers are written as *minimum* acceptable responses rather than exhaustive ideal answers, so a model response that covers the required facts is scored as correct even if it includes additional relevant information.

3.3.3 Evaluation Environment

Evalutation Framework:
The evaluation will be conducted using **DeepEval** [Con23], a framework for evaluating LLM outputs against reference answers. DeepEval allows for automatic scoring based.

Agent Configurations:
Two agent configurations are evaluated:

- **Custom Agent**: The full agent configuration with access to all tools, including structured factsheet access (`show_elements`, `list_attachments`), direct file reading (`read_attachments`), and RAG capabilities.
- **Baseline + RAG Agent**: An ablated version with access to web search, law retrieval, and project document RAG, but without direct file access or structured factsheet tools.

A web-search-only baseline was considered but excluded from the evaluation: without access to the case documents it consistently exhausted the model's context window before producing meaningful responses, making a fair comparison impossible.

Model Configuration:
Since the scope of the thesis is not to evaluate the models themselves, but rather the design and architecture of the agent, a selection of four models is used for comparison. These include both closed-source and open-source variants to ensure the agent's performance is consistent across different provider architectures.

Table 3.2: Overview of Large Language Models used in evaluation.

Model Name	Type	Total Params	Active Params	Provider
Claude-sonnet-4-6	Closed	Unknown	Unknown	Anthropic
Gemini 2.5 Flash	Closed	Unknown	Unknown	Google
Qwen 3.5 [YLY+25]	Open (MoE)	397B	17B	Alibaba
Qwen 3 Next Instruct [YLY+25]	Open (MoE)	80B	3B	Alibaba

The evaluation will be conducted across all datasets using consistent model-agent pairings. To account for the inherent stochastic nature of LLM outputs, each evaluation set (model + agent configuration) will be executed 5 times to ensure statistical significance in the results.

3.3.4 Metrics

The evaluation will focus on several key metrics, measering both the quality of the agent's responses and its efficiency in terms of resource usage. Metrics that are to be evaluated are:

- **Correctness, Relevancy, and Completeness**: These semantic metrics are evaluated using the DeepEval framework. The agent's output is compared against a "gold standard" reference created in collaboration with legal experts. Each metric is scored on a scale from 0 to 1.
- **Self-consistency**: The degree to which the agent's output remains consistent across multiple queries and over time as the case evolves.

$$\text{Self-Consistency} = \frac{2}{n(n-1)} \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \cos(\mathbf{e}_i, \mathbf{e}_j) \tag{3.1}$$

where n is the number of generated outputs, $\mathbf{e}_i$ and $\mathbf{e}_j$ are the sentence embeddings of the i-th and j-th output, and the cosine similarity is defined as:

$$\cos(\mathbf{e}_i, \mathbf{e}_j) = \frac{\mathbf{e}_i \cdot \mathbf{e}_j}{\|\mathbf{e}_i\| \|\mathbf{e}_j\|} \tag{3.2}$$

- **BERTScore**: A metric that evaluates the similarity between the agent's output and a reference text using BERT embeddings [ZKW+20].
- **Passrate**: The percentage of queries for which the agent's output meets a predefined quality threshold.
- **Efficiency Metrics**: **Token Usage** (total tokens consumed) and **Inference Time** (latency) are recorded to assess the practical viability of the system.

Correctness, relevancy, Completeness are all evaluated against the expected out (constructed in collaboration with legal experts) using DeepEval. The framework rates each response on a scale from 0 to 1, with a certain threshold for what is considered a "pass" for the passrate metric.

BERTScore is calculated based on the expected output. Self-consistency is evaluated by comparing the agent's responses across multiple queries and sessions to assess whether it maintains a coherent understanding of the case over time. Token usage and inference time are recorded during the agent's operation.

3.3.5 One-sided Hypothesis Tests

For quality/performance metrics (correctness, relevancy, completeness, passrate, self-consistency, BERTScore):

$$H_0 : \mu_{\text{custom}} \leq \mu_{\text{baseline+RAG}} \qquad H_1 : \mu_{\text{custom}} > \mu_{\text{baseline+RAG}}$$

For cost/efficiency metrics (token usage, inference time):

$$H_0 : \mu_{\text{custom}} \geq \mu_{\text{baseline+RAG}} \qquad H_1 : \mu_{\text{custom}} < \mu_{\text{baseline+RAG}}$$

Notation:

- μ_{custom} = population mean for the custom agent
- $\mu_{\text{baseline+RAG}}$ = population mean for the baseline with RAG agent

All tests are one-tailed / one-sided.

Chapter 4

Results

In evaluating the performance, it is important to note that the datasets used were specifically developed and reverse-engineered for this study to simulate real-world cases. A key uncertainty remains whether these synthetic datasets are sufficiently complex or large enough to reach the "tipping point" where the custom architecture's strengths truly manifest. Therefore, the evaluation should prioritize the performance trends across sessions as a case progresses, rather than focusing solely on aggregate performance metrics.

The fundamental trade-off addressed in this research is cost versus accuracy. While baseline models—which ingest the full context for every query—may offer higher granular precision in smaller datasets, this "stateless agent" approach is not scalable. In larger, multi-year cases, passing the entire history becomes prohibitively expensive and computationally slow. The custom architecture accepts a marginal risk in retrieval error to achieve sustainable operational costs and long-term scalability.

A human analogy is helpful to illustrate the architectural philosophy: A legal professional working on a massive case does not re-read every single document in the archive before answering a new question. Instead, they rely on a structured "factsheet" or summary (the case state) and perform targeted lookups when specific details are required. While reading every page every time might provide the most detail, it eventually leads to cognitive overload where one "mixes up the cards." By utilizing a condensed case state, the custom architecture avoids this "context confusion" while maintaining a manageable overhead.

Consequently, for the smaller datasets created for these tests, the baseline models often perform better because the room for retrieval error is eliminated when everything fits in the prompt. However, as the volume of information reaches realistic proportions, the necessity of the "factsheet" approach becomes clear. The extraction process not only manages costs but also yields a structured, machine-readable state that can be seamlessly integrated into front-end systems for human oversight, bridging the gap between raw data and actionable legal insights.

Chapter 5

Discussion

In progress

Chapter 6

Conlusion

In progress

Bibliography

[Con23] Confident AI. Deepeval: The llm evaluation framework, 2023.

[HSK+24] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. Ruler: What's the real context size of your long-context language models? *arXiv preprint arXiv:2404.06654*, 2024.

[JWL+23] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Llmlingua: Compressing prompts for accelerated inference of large language models. *arXiv preprint arXiv:2310.05736*, 2023.

[JWL+24] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Longllmlingua: Accelerating and enhancing llms in long context scenarios via prompt compression. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1658–1677, 2024.

[Lan24] LangChain. Langgraph, 2024. Accessed: 2025-11-26.

[LLH+23] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts, 2023.

[LPP+21] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks, 2021.

[LWC+25] Hao Liu, Zhengren Wang, Xi Chen, Zhiyu Li, Feiyu Xiong, Qinhan Yu, and Wentao Zhang. Hoprag: Multi-hop reasoning for logic-aware retrieval-augmented generation. *arXiv preprint arXiv:2502.12442*, 2025.

[MFG24] Tsendsuren Munkhdalai, Manaal Faruqui, and Siddharth Gopal. Leave no context behind: Efficient infinite context transformers with infini-attention. *arXiv preprint arXiv:2404.07143*, 101, 2024.

[Mic24] Microsoft. Semantic kernel python: Context management, 2024. Accessed: 2025-11-26.

[Ram18] Sebastián Ramírez. Fastapi, 2018. Accessed: 2025-11-26.

[Sup20] Supabase. Supabase, 2020.

[VSP+23] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023.

[XSC23] Fangyuan Xu, Weijia Shi, and Eunsol Choi. Recomp: Improving retrieval-augmented lms with compression and selective augmentation. *arXiv preprint arXiv:2310.04408*, 2023.

[YLY+25] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai

Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025.

[ZKW+20] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. Bertscore: Evaluating text generation with bert, 2020.